

Open-Source AI Agent CLIs — Feature Analysis & Comparison Report

Generated: 2026-02-26 | Sources: 42 KB items across 4 crawled repos + web research
Knowledge Base Sources: OpenCode (262), Crush (263), awesome-opencode (264), Codex (265)

1. Executive Summary

The AI coding agent CLI landscape in early 2026 has matured into a competitive ecosystem with ~15 notable tools. Key trends:

- Go and Rust dominate** new CLI implementations (OpenCode, Crush, Codex-rs, Goose), displacing Python/TypeScript for performance
- MCP (Model Context Protocol)** is the universal extensibility standard — every major tool supports it or plans to
- Multi-provider support** is table stakes — tools support 7-12+ LLM providers via abstraction layers
- LSP integration** separates serious tools from toys — only OpenCode, Crush, and Zed have deep LSP support
- Sandboxing** is the new frontier — Codex leads with Seatbelt/Landlock, others rely on permission prompts
- Agent-as-server** patterns emerging — Codex can act as an MCP server, enabling meta-agent architectures
- TUI frameworks converge** on Bubble Tea (Go) and Ratatui (Rust) for rich terminal interfaces

Market Positioning (Feb 2026)

Tier	Tools	Stars
Dominant	Gemini CLI (~95K), Zed (~76K), Cline (~58K), Codex (~49K), Claude Code (~41K), Aider (~41K)	50K+
Strong	Continue (~32K), Goose (~31K), OpenCode (~30K), Warp (~26K)	25-50K
Rising	Qwen Code (~15K), Plandex (~15K), Crush (~14K), Codebuff (~3K)	<15K

2. Per-Tool Deep Dives

2.1 OpenCode (opencode-ai/opencode)

Overview: Go-based terminal AI coding assistant with Bubble Tea TUI. ~29.7K stars, Apache-2.0.

Architecture:

- `cmd/` → CLI endpoint
- `internal/app` → service orchestration
- `internal/llm` → agent loop + providers + tools + prompts
- `internal/tui` → Bubble Tea UI
- `internal/lsp` → Language Server Protocol integration
- `internal/db` → SQLite persistence via `sqlc`
- Config: Viper-based, `~/.opencode.json` OR `./.opencode.json`

Agent Loop: Single-thread ReAct loop. `processGeneration()` enters tool-use loop, executes tools sequentially with permission checks. Loop continues while finish reason is `ToolUse`. Cancellation via `sync.Map` keyed by `sessionID`.

Tools (12+):

Tool	Description
bash	Persistent shell session, 60s default / 600s max timeout. Safe commands bypass permission. Banned: curl, wget, nc, telnet, browsers
edit	Line-range text replacement with conflict detection
write	File creation/update, auto-creates dirs, checks mod time vs last read
view	File reading with line ranges
patch	Unified diff patching with fuzz detection (rejected if fuzz > 3), atomic application
glob	File pattern matching
grep	Dual strategy: ripgrep first, regex fallback. 100 file limit, 200 match cap
ls	Directory listing
fetch	HTTP fetching
sourcegraph	Cross-repo code search via Sourcegraph API
agent	Sub-agent spawning (read-only tool subset: glob, grep, ls, sourcegraph, view)
diagnostics	LSP diagnostics (when LSP available)

Provider Support: 7+ providers — OpenAI, Anthropic, Google, Azure, AWS Bedrock, Groq, OpenRouter, plus OpenAI-compatible APIs.

MCP: Client-side MCP support for external tool servers.

LSP Integration: Deep integration via `internal/lsp`. Diagnostics tool queries language servers. Write and patch tools wait for LSP diagnostics after file changes to catch syntax errors.

TUI: Bubble Tea framework with Lip Gloss styling. Multi-pane layout.

Session Management: SQLite-backed via `sqlc`. Session persistence, message history, title auto-generation.

Permission System: `internal/permission` package. Safe read-only commands (`git status`, `go test`, `ls`) bypass approval. File operations require read-before-edit. Permission denial stops remaining tool executions.

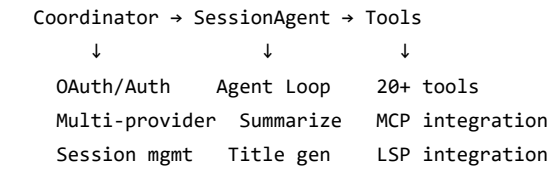
Unique Features:

- Sourcegraph integration for cross-repo search
- Sub-agent with restricted read-only tool set
- LSP diagnostics integrated into write/patch workflows
- Viper-based layered config (user + project)

2.2 Crush (charmbracelet/crush)

Overview: Terminal AI coding agent by Charmbracelet. Go 1.26, ~14.4K stars. Built on the Fantasy framework (`charm.land/fantasy`).

Architecture:



- `coordinator.go` — top-level orchestrator, manages sessions, builds agents
- `agent.go` — agent loop per session, message queuing, auto-summarization
- `tools/*.go` — 20+ built-in tools via Fantasy AgentTool interface
- `tools/mcp/` — full MCP support (stdio/HTTP/SSE transports)
- `hyper/provider.go` — Charmbracelet's own LLM proxy (hyper.charm.land)
- `prompt/` — Go text/template system, compile-time embedded templates

Tools (20+):

Tool	Description
bash	Shell execution with background support. Auto-backgrounds after 60s. Blocklist: curl, wget, ssh, sudo, apt, brew, etc.
edit	Single file edit with read-before-edit enforcement, mod time checks
multi_edit	Batch sequential edits on a single file
view	File reading, appends LSP diagnostics when viewing
glob	File pattern matching
grep	Content search
ls	Directory listing
write	File creation
fetch	HTTP fetching (5MB limit, 600s timeout, http/https only)
download	File download
web_search	DuckDuckGo search (no API key needed)
sourcegraph	Cross-repo search via Sourcegraph GraphQL API (max 20 results)
agent	Sub-agent spawning for delegated tasks
agentic_fetch	Dedicated sub-agent for web research with its own tool set
diagnostics	LSP diagnostics (file-specific + project-wide, sorted by severity)
references	LSP FindReferences with grep pre-filter, qualified symbol support
lsp_restart	Restart specific or all LSP clients
job_output	Retrieve stdout/stderr from background shells
job_kill	Terminate background shells
todos	Built-in task tracking (pending/in_progress/completed)
list_mcp_resources	List MCP server resources
read_mcp_resource	Read specific MCP resource

Provider Support: Multi-provider via Fantasy framework — OpenAI, Anthropic, Google, Azure, Bedrock, plus Hyper (Charmbracelet's own proxy at hyper.charm.land). Hyper supports JSON and streaming modes, retry with Retry-After, credit tracking.

MCP Integration (Full):

- Three transports: stdio, HTTP (StreamableClient), SSE
- Four states: Disabled → Starting → Connected → Error (published via pubsub)
- Full support: tools, resources, prompts
- Tool names: `mcp_[serverName]_[toolName]`
- Permission checks before every MCP tool execution
- Image/media capability validation per model

LSP Integration (Deep):

- `lsp.Manager` manages multiple language server clients
- Diagnostics: severity-sorted, 10 per category cap, file-specific + project-wide
- References: grep pre-filter → LSP FindReferences, qualified symbol support (`::`, `.`, `\`)
- LSP restart with concurrent goroutines
- Configured in `crush.json` (Go, TypeScript, Nix, any LSP server)

TUI: Full Charm stack — Bubble Tea (framework), Lip Gloss (styling), Glamour (markdown), Bubbles (components).

Session Management: Coordinator manages sessions. Auto-summarization, title generation. Session-based permission context.

Permission System: Multi-layered:

- Bash: command blocklist + argument blockers + safe whitelist
- Files: read-before-edit, mod time check, symlink resolution, out-of-workdir permission
- MCP: per-tool permission, disabled tool filtering
- Global: `--yo1o` flag skips all checks, `crush.json` allowlists

Loop Detection: SHA-256 hash of (tool name + input + output) over sliding window of 10 steps. Triggers if any signature repeats 5+ times.

Prompt System: Go text/template with compile-time embedding. Three templates: coder (main), task (sub-agent), initialize. Injects git status, branch, recent commits, skills XML, context files.

Skills System: Discoverable packages from `~/.config/crush/skills/` and custom paths. Serialized as XML into prompts.

Unique Features:

1. Hyper provider (Charmbracelet's LLM proxy)
2. Sourcegraph cross-repo search
3. Agentic fetch (dedicated web research sub-agent)
4. Background jobs with auto-background (60s threshold)
5. Built-in todo tracking
6. DuckDuckGo search (no API key)
7. Multi-edit tool
8. Fantasy framework (charm.land/fantasy)
9. `.crushignore` support
10. [AGENTS.md](#) project context

2.3 OpenAI Codex CLI (openai/codex)

Overview: Dual-layer architecture — legacy TypeScript CLI (superseded) and current Rust CLI. ~49.3K stars, Apache-2.0.

Architecture:

```
Rust workspace (codex-rs/):
  core/      - business logic library
  exec/      - headless CLI
  tui/       - Ratatui-based TUI
  cli/       - multitool binary
  app-server-protocol/ - JSON-RPC schema
```

Multiple deployment surfaces: CLI, IDE extension (VS Code/Cursor/Windsurf), Codex App (macOS desktop), Codex Web (chatgpt.com/codex). Distributed via npm ([@openai/codex](https://github.com/openai/codex)), Homebrew, GitHub Release binaries.

Agent Loop: ReAct-style think-act-observe cycle in `AgentLoop` . Structured output constraints prevent hallucination. Self-healing: stderr fed back as context. Token optimization: truncation, sliding window (3-5 interactions), batching.

Tools (minimal but powerful):

Tool	Description
<code>apply_patch</code>	Virtual CLI via <code>codex-arg0</code> crate, unified diff patches
<code>local_shell_call</code>	Sandboxed shell with command array, cwd, env, timeout
<code>web_search_call</code>	Web search
<code>function_call</code>	Standard OpenAI function calling
<code>custom_tool_call</code>	MCP/dynamic tools via <code>DynamicToolSpec</code>

Provider Support (10+): OpenAI (default, o4-mini), OpenRouter, Azure, Gemini, Ollama, Mistral, DeepSeek, xAI, Groq, ArceeAI, plus any OpenAI-compatible API. Uses Responses API.

MCP (Dual):

- Client: connects to servers from `config.toml`
- Server: `codex mcp-server` (experimental) — allows other agents to use Codex as a tool
- Management: `codex mcp add/list/get/remove`
- OAuth support for MCP servers

LSP: No native LSP integration.

TUI: Ratatui-based terminal UI (Rust).

Session Management: SQLite state DB (`CODEX_SQLITE_HOME`). Ghost commits for invisible repo snapshots. Compaction with encrypted content. Plan mode with separate reasoning effort. `codex exec --ephemeral` for no-persist runs.

Sandbox Model (Best-in-class):

- macOS: Apple Seatbelt (`sandbox-exec`) — read-only jail, configurable writable roots, network fully blocked
- Linux: Landlock LSM via `codex-arg0` crate self-re-execution; Docker recommended for stronger isolation
- Windows: WSL2 only
- Three modes: read-only (default), workspace-write, danger-full-access
- Full Auto = network-disabled + directory-confined

Permission System: `AskForApproval` enum: `untrusted`, `on-failure`, `on-request`, `never`, `reject`.

App-Server Protocol: JSON-RPC between frontends and core backend. Key types: `InitializeParams`, `CommandExecParams`, `ConfigRead/Write`, `FuzzyFileSearch`, `ReviewStartParams`. Enums: `SandboxMode`, `AskForApproval`, `ReasoningEffort` (`none`→`xhigh`), `Personality` (`none/friendly/pragmatic`).

Config: TOML at `~/.codex/` . Layered: user → project. [AGENTS.md](#) hierarchy: `~/.codex/AGENTS.md` → repo root → cwd. External agent config detection/import.

Skills System: Filesystem-based via `.codex/skills/<name>/` . [SKILL.md](#) with YAML frontmatter. Built-in: babysit-pr (autonomous PR monitoring), test-tui. Codex App extends skills into Automations (scheduled background jobs).

Unique Features:

1. Ghost commits (invisible repo snapshots)
2. Babysit-pr skill (autonomous DevOps agent with CI failure classification)
3. Codex as MCP server (meta-agent architectures)
4. External agent config migration
5. Personality system (none/friendly/pragmatic)
6. Built-in code review (uncommittedChanges/baseBranch/commit/custom targets)
7. Plan mode with configurable reasoning effort
8. Rust native performance
9. Best-in-class sandboxing (Seatbelt + Landlock)
10. Multiple deployment surfaces (CLI, IDE, App, Web)

2.4 Claude Code (Anthropic)

Overview: Anthropic's official CLI for Claude. TypeScript, ~40.8K stars. The reference implementation for AI coding agents.

Architecture: Node.js CLI with hooks system, MCP integration, skills, and sub-agent spawning via Task tool.

Tools (10+): Read, Edit, Write, Glob, Grep, Bash, Task (sub-agents), WebFetch, WebSearch, NotebookEdit, AskUserQuestion, EnterPlanMode, plus MCP tools.

Provider Support: Anthropic models only (Claude Opus 4.6, Sonnet 4.6, Haiku 4.5). Model routing: Opus for architecture, Sonnet for features, Haiku for quick tasks.

MCP: Full client support. Tools, resources, prompts. Configured in settings.json.

LSP: No native LSP (relies on IDE integration when used as extension).

TUI: Terminal chat interface with markdown rendering.

Session Management: JSONL transcript files. Context compaction at ~120K tokens. Pre-compact hooks for continuity.

Permission System: Permission modes with tool-level approval. Hooks can intercept and allow/deny/ask. [CLAUDE.md](#) for project instructions.

Hooks System (14 events): SessionStart, UserPromptSubmit, PreToolUse, PermissionRequest, PostToolUse, PostToolUseFailure, Notification, SubagentStart, SubagentStop, Stop, TeammateIdle, TaskCompleted, PreCompact, SessionEnd. Hooks can inject context, block actions, modify permissions.

Unique Features:

1. Most mature hooks/lifecycle system (14 events)
2. Sub-agent spawning with isolated context (Task tool)
3. Skills system (user-invocable slash commands)
4. [CLAUDE.md](#) project instructions (hierarchical)
5. Context compaction with pre-compact hooks
6. Permission modes with granular tool-level control
7. Notebook editing support
8. Plan mode with user approval flow

2.5 Gemini CLI (Google)

Overview: Google's terminal AI agent. TypeScript, ~95.7K stars (highest in category). Apache-2.0.

Key Specs:

- 1M token context window (largest by far)
- Gemini 2.5 Pro / Gemini Flash (Google models only)
- MCP support (local and remote servers)
- Conversation checkpointing (save/resume)
- Trusted Folders security system
- [GEMINI.md](#) project config
- Multimodal input (sketches, PDFs)
- Google Search grounding
- Free tier: 60 req/min, 1K req/day

Unique Features: Massive context window, free tier generosity, Google Search integration, multimodal input.

2.6 Aider

Overview: AI pair programming CLI. Python, ~40.9K stars. Apache-2.0. Pioneer of the repo-map concept.

Key Specs:

- Tree-sitter repo map for codebase understanding
- Automatic git commits as checkpoints
- Voice-to-code
- IDE watch mode
- Linting/testing with auto-fix
- 100+ language support
- Edit formats: whole/diff/udiff
- Architect mode (two-model workflow)
- SWE-bench leader
- No native MCP (community fork mcpm-aider exists)
- No native LSP (uses tree-sitter)
- Model support: Claude, DeepSeek, GPT-4o, o1, o3-mini, virtually any LLM via Ollama

Unique Features: Pioneered repo-map, git-as-session-management, architect mode, voice-to-code, broadest language support.

2.7 Goose (Block/Square)

Overview: Extensible AI agent. Rust (59%) + TypeScript (33%), ~31.2K stars. Apache-2.0.

Key Specs:

- Extensions ARE MCP servers (first-class MCP architecture)
- CLI + Desktop GUI app
- Any LLM provider, multiple configs simultaneously
- Recipes system for composable workflows
- Mentor mode for learning
- 113 releases (v1.25.0)
- Session-based conversation history

Unique Features: Extensions-as-MCP-servers architecture, dual CLI+desktop, recipes, mentor mode.

2.8 Additional Notable Tools

- Cline (~58.4K stars)** — VS Code extension. Autonomous multi-step execution, browser automation via Playwright, Computer Use capability, workspace checkpoints, MCP tool creation on request. Forks: Roo Code, Kilo Code.
- Continue.dev (~31.5K stars)** — VS Code/JetBrains extension + CLI. AI checks as GitHub status checks, checks-as-code in `.continue/checks/`, MCP Registry integration.
- Warp (~26K stars)** — GPU-accelerated terminal replacement. Multi-agent orchestration (Oz agent), supports CLI agents (Claude Code, Codex, Gemini CLI). \$200/month enterprise.
- Zed (~75.9K stars)** — Rust editor with Agent Panel, Zeta autocomplete model, ACP (Agent Client Protocol) for third-party agents, multiplayer editing. Full LSP support as IDE.
- Amp/Sourcegraph** — Closed source. Multi-model auto-routing, Deep mode, Oracle/Librarian sub-agents, Sourcegraph code intelligence backbone, shared threads.
- Codebuff (~2.9K stars)** — Multi-agent architecture (File Picker → Planner → Editor → Reviewer), [knowledge.md](#) memory, SDK for embedding. Claims 61% vs Claude Code's 53% on evals.
- Plandex (~14.6K stars)** — Go, massive context (20M+ tokens), sandbox execution.
- Qwen Code (~14.9K stars)** — Gemini CLI fork optimized for Qwen3-Coder.

3. Feature Comparison Matrix

Core Features

Feature	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Language	Go	Go	Rust+TS	TypeScript	TypeScript	Python	Rust+TS
Stars	~30K	~14K	~49K	~41K	~96K	~41K	~31K
License	Apache-2.0	Apache-2.0	Apache-2.0	Proprietary	Apache-2.0	Apache-2.0	Apache-2.0
TUI Framework	Bubble Tea	Bubble Tea	Ratatui	Custom	Custom	Terminal	CLI+Desktop

Agent Loop & Tools

Feature	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Agent Loop	ReAct single	ReAct single	ReAct single	ReAct single	ReAct single	ReAct single	ReAct single
Built-in Tools	12+	20+	2 core + 3	10+	~8	~6	MCP-based
Sub-agents	Yes (read-only)	Yes (agentic fetch)	No	Yes (Task tool)	No	No (architect mode)	No
Background Jobs	No	Yes (auto-bg 60s)	No	No	No	No	No

Feature	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Loop Detection	No	Yes (SHA-256 window)	No	Yes (hooks)	No	No	No
Todo Tracking	No	Yes (built-in)	No	Yes (TaskCreate)	No	No	No

Provider & Model Support

Feature	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Providers	7+	Multi + Hyper	10+	Anthropic only	Google only	Any LLM	Any LLM
Local Models	Via OpenAI-compat	Via providers	Ollama	No	No	Ollama	Ollama
Model Routing	No	No	No	Yes (Opus/Sonnet/Haiku)	No	Architect mode	Multiple configs
Own Proxy	No	Hyper (charm.land)	No	No	No	No	No

MCP & LSP

Feature	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
MCP Client	Yes	Yes (full)	Yes	Yes	Yes	No	Yes (first-class)
MCP Server	No	No	Yes (experimental)	No	No	No	No
MCP Transports	stdio	stdio/HTTP/SSE	stdio	stdio	stdio	—	stdio
MCP Resources	No	Yes	No	Yes	No	—	Yes
MCP Prompts	No	Yes	No	Yes	No	—	No
LSP Diagnostics	Yes	Yes (deep)	No	No	No	No	No
LSP References	No	Yes	No	No	No	No	No
LSP Restart	No	Yes	No	No	No	No	No

Session & Config

Feature	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Persistence	SQLite	Session storage	SQLite	JSONL files	Checkpoints	Git commits	Session files
Config Format	JSON	JSON (crush.json)	TOML	JSON	GEMINI.md	CLI flags/YAML	YAML
Config Layers	User + Project	User + Project	User + Project	Global + Project	Project	CLI + config	Project
Project File	.opencode.json	AGENTS.md	AGENTS.md	CLAUDE.md	GEMINI.md	.aider*	GOOSE.md
Context Window	Standard	Auto-summarize	Sliding window	120K + compaction	1M tokens	Repo map	Standard
Auto-summarize	Yes	Yes	Yes (compaction)	Yes (compaction)	No (huge context)	No	No

Security & Permissions

Feature	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Permission System	Yes	Yes (multi-layer)	Yes (sandbox)	Yes (modes+hooks)	Trusted Folders	No	Governance
Sandbox	No	No	Yes (Seatbelt/Landlock)	No	Yes	No	No
Command Blocklist	Yes (curl,wget,nc)	Yes (extensive)	N/A (sandboxed)	No (hook-based)	No	No	No
Read-before-edit	Yes	Yes	N/A	Yes	No	No	No
Skip-all Flag	No	--yolo	danger-full-access	No	No	No	No
Mod Time Check	Yes	Yes	No	No	No	No	No

Extensibility

Feature	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Skills/Plugins	No	Yes (skills dir)	Yes (SKILL.md)	Yes (slash commands)	No	No	Recipes
Hooks/Lifecycle	No	No	No	Yes (14 events)	No	No	No
Web Search	No	DuckDuckGo	Yes	WebSearch tool	Google Search	No	Via MCP
Sourcegraph	Yes	Yes	No	No	No	No	No

Feature	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Git Integration	Basic	Yes (attribution)	Ghost commits	Via hooks	Basic	Deep (auto-commit)	Basic
Code Review	No	No	Yes (built-in)	No	No	No	No
Diff System	Unified diff	Edit + MultiEdit	apply_patch (unified)	Edit (string replace)	Unknown	whole/diff/udiff	Unknown

4. Best-in-Class Analysis

Category	Best Tool	Why
Sandboxing	Codex	Only tool with OS-level sandboxing (Seatbelt on macOS, Landlock on Linux). Network fully blocked in auto mode. Others rely on permission prompts.
LSP Integration	Crush	Deepest LSP: diagnostics, references, restart. Multi-client manager. View tool auto-appends diagnostics. OpenCode has diagnostics only.
MCP Support	Crush / Goose	Crush: full spec (tools+resources+prompts), 3 transports, state machine. Goose: extensions ARE MCP servers. Codex unique as MCP server.
Tool Breadth	Crush	20+ tools including background jobs, todo tracking, multi-edit, agentic fetch, DuckDuckGo, Sourcegraph. Most comprehensive built-in toolkit.
Multi-Provider	Codex	10+ built-in providers plus any OpenAI-compatible API. Broadest out-of-box support.
Context Management	Gemini CLI	1M token window eliminates need for compaction/summarization. Claude Code's hook-based compaction is most sophisticated for smaller windows.
Hooks/Lifecycle	Claude Code	14 lifecycle events with full interception. No other tool comes close. Crush and OpenCode have zero hook support.
Sub-agents	Claude Code	Task tool with isolated context, model selection (Opus/Sonnet/Haiku), parallel execution. Crush has agentic fetch. OpenCode has read-only sub-agent.
TUI Quality	Crush / OpenCode	Both use Bubble Tea (Go's best TUI framework). Crush adds Lip Gloss + Glamour for styling/markdown. Codex uses Ratatui (Rust equivalent).
Git Integration	Aider	Auto-commits as checkpoints, git-as-session-management. Codex has ghost commits. Others are basic.
Repo Understanding	Aider	Pioneered tree-sitter repo maps. AST-based context selection across 100+ languages. Others use grep/glob.
Security Model	Codex	OS-level sandbox + approval modes + network blocking. Crush has best permission-based model (multi-layer, mod time checks).
Skills/Extensibility	Claude Code	Skills + hooks + MCP = most extensible. Codex has SKILL.md + Automations. Crush has skills dir.
Background Execution	Crush	Only tool with auto-background (60s threshold), job tracking, job kill. Essential for dev servers and long builds.
Code Review	Codex	Built-in review with multiple targets (uncommitted, branch, commit, custom). No other CLI has native code review.

Category	Best Tool	Why
Cross-repo Search	OpenCode / Crush	Both integrate Sourcegraph. Crush also has DuckDuckGo. Others lack cross-repo capability.
Loop Detection	Crush	SHA-256 signature hashing over sliding window. Simple, effective. Claude Code does it via hooks (more flexible but external).
Prompt Engineering	Crush	Go template system with compile-time embedding, git context injection, skills XML serialization. Most structured approach.
Performance	Codex	Rust native binary. Fastest startup and execution. Go tools (OpenCode, Crush) are second tier. TS/Python tools are slowest.
Free Tier	Gemini CLI	60 req/min, 1K req/day free. No other tool offers comparable free access.

5. Recommendations for cc-cli

Based on this analysis, the following features represent the highest-value implementations for a new CLI agent:

Must-Have (Table Stakes)

- Multi-provider abstraction** — Support 5+ providers minimum (OpenAI, Anthropic, Google, local via Ollama, OpenRouter). Use a provider interface pattern like OpenCode/Crush.
- MCP client support** — Full spec: tools + resources + prompts. Support stdio transport minimum, HTTP/SSE for remote servers. Follow Crush's state machine pattern (Disabled→Starting→Connected→Error).
- Permission system** — Multi-layer like Crush: command blocklist, safe whitelist, read-before-edit enforcement, mod time checks. Add a `--yo1o` equivalent for power users.
- Session persistence** — SQLite-backed (like OpenCode/Codex). Store messages, tool calls, metadata. Support session resume.
- Project config file** — [AGENTS.md](#) or equivalent with hierarchical loading (global → repo → cwd).

High-Value Differentiators

- LSP integration** — Follow Crush's model: diagnostics + references + restart. Multi-client manager. Auto-append diagnostics on file view. Run diagnostics after writes/patches. This is a major differentiator — only 2 of 15 tools have it.
- Hooks/lifecycle system** — Claude Code's 14-event system is the gold standard. Start with 5-6 core events: SessionStart, PreToolUse, PostToolUse, PreCompact, Stop. Enable context injection and permission interception.
- Background job management** — Crush's auto-background pattern (60s threshold) is elegant. Essential for dev servers, builds, test suites. Include `job_output` and `job_kill`.
- Loop detection** — Crush's SHA-256 sliding window is simple and effective. 10-step window, 5-repeat threshold.
- Sub-agent spawning** — Isolated context windows for parallel research. Read-only tool subset for safety. Claude Code's Task tool is the reference.

Nice-to-Have (Competitive Edge)

- Sourcegraph integration** — Cross-repo search is powerful for understanding library usage patterns. Both OpenCode and Crush have it. GraphQL API, straightforward to implement.
- Codex-as-MCP-server pattern** — Exposing the agent itself as an MCP server enables meta-agent architectures. Unique to Codex, high innovation value.
- OS-level sandboxing** — Seatbelt (macOS) + Landlock (Linux) like Codex. Significantly stronger than permission prompts. Complex to implement but best-in-class security.
- Built-in code review** — Codex's review with multiple targets (uncommitted, branch, commit). Natural workflow integration.
- Skills/plugin system** — Filesystem-based like Codex ([SKILL.md](#)) or Crush (skills dir). Enable community extensibility.

16. **Todo/task tracking** — Crush's built-in todo system gives users visibility into multi-step operations. Simple to implement, high UX value.
17. **Agentic fetch** — Crush's dedicated web research sub-agent with its own tool set. Better than raw fetch for research tasks.
18. **Ghost commits** — Codex's invisible repo snapshots for rollback. Elegant undo mechanism.
19. **Auto-summarization** — Both OpenCode and Crush auto-summarize sessions. Essential for context management with smaller windows.
20. **Prompt template system** — Crush's Go template approach with compile-time embedding, git context injection, and skills serialization. Structured and maintainable.

Architecture Recommendations

- **Language:** Go or Rust. Go has better ecosystem for TUI (Bubble Tea) and faster development. Rust for maximum performance and sandboxing.
- **TUI:** Bubble Tea (Go) or Ratatui (Rust). Both are mature, well-documented.
- **DB:** SQLite for sessions, config, and state. Proven by OpenCode, Codex, and our memory-tools.
- **Config:** TOML (like Codex) or JSON (like OpenCode). Layered: user → project.
- **Agent loop:** Single-thread ReAct with tool-use continuation. Sequential tool execution with permission checks.
- **Provider interface:** Abstract provider with Generate() and Stream() methods. Functional options pattern (like Crush's Hyper).

6. Knowledge Base References

Source IDs

Source	ID	Items	Description
OpenCode	262	10	Architecture, tools, agent loop, providers
Crush	263	14	Architecture, tools, MCP, LSP, permissions, prompts, unique features
awesome-opencode	264	10	Landscape overview: Gemini CLI, Aider, Goose, Cline, Continue, Warp, Zed, Amp, Codebuff
Codex	265	8	Architecture, tools, sandbox, MCP, config, skills, protocol, unique features

Key KB Item IDs by Topic

OpenCode:

- 8368 — Architecture overview
- 8369 — Agent loop
- 8370 — Tools system
- 8371 — Bash tool
- 8373 — Patch tool
- 8375 — Write tool
- 8376 — Grep tool

Crush:

- 8396 — Architecture overview
- 8399 — Loop detection
- 8400 — Hyper provider
- 8401 — MCP integration
- 8402 — Prompt system
- 8403 — Background jobs

- 8404 — LSP integration
- 8405 — Sourcegraph
- 8406 — Todo system
- 8407 — Security & permissions
- 8409 — Unique features comparison

Codex:

- 8388 — Architecture overview
- 8389 — App-Server Protocol
- 8390 — Sandbox model
- 8391 — Skills system
- 8392 — Tools & agent loop
- 8393 — Multi-model & MCP
- 8394 — Config & session
- 8395 — Unique features & strengths/weaknesses

Landscape (awesome-opencode):

- 8378 — Aider
- 8379 — Goose
- 8380 — Gemini CLI
- 8381 — Cline
- 8382 — Continue.dev
- 8383 — Amp/Sourcegraph
- 8384 — Codebuff
- 8385 — Zed
- 8386 — Warp

Search Queries for KB Retrieval

```
mem_kb({action:"search", query:"OpenCode tools architecture"})
mem_kb({action:"search", query:"Crush MCP LSP permissions"})
mem_kb({action:"search", query:"Codex sandbox skills protocol"})
mem_kb({action:"search", query:"AI agent CLI comparison landscape"})
```

Report generated from 42 KB items across 4 crawled repositories and web research. All data stored in memory-tools vector DB for future reference.